

8-2 · DAG 이해 및 유즈케이스 연구 · 실습 1H

유즈케이스 연구와 태스크 분해 설계

실무 사례 다섯을 훑고 우리 파이프라인을 짤 건다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 실무 유즈케이스 다섯의 공통점을 한 문장으로 말한다
- ② 우리 파이프라인을 태스크 5개로 나누고 먹등 여부를 적는다
- ③ pipeline_skeleton 골격 DAG 를 만들어 Graph 모양을 맞춘다
- ④ 태스크 분해안 문서를 채워 8-3 의 출발점으로 남긴다

여기서 만든 골격과 문서가 8-3 실습의 설계도이자 8주차 산출물의 일부입니다

다섯 단계 — 사례에서 골격 DAG 와 문서까지

7주 동안 만든 것을 태스크로 쪼갬니다 — 팀별로 다르게 나눠도 됩니다, 근거만 있으면

1. 유즈케이스	2. 태스크 분해	3. 골격 DAG	4. 실행 확인	5. 문서화
실무 사례 5 공통점 찾기	5개로 쪼개고 먹등한가	print 만 모양 먼저	unpause·trigger dags show	분해안 표 근거 한 줄

실무 유즈케이스 다섯 — Airflow 가 하는 일

유즈케이스	주기	Airflow 가 하는 일
배치 ETL(야간 정산)	매일 새벽	수집→정제→적재 순서 보장, 실패 단계만 재시도
ML 모델 재학습	매주	데이터 준비→학습→평가→배포를 조건부로
리포트 자동 발송	매일 아침	집계 완료를 기다렸다가 생성·발송
데이터 품질 점검	적재 후	건수·널·중복 검사, 미달이면 다음 단계 중단
RAG 인덱스 갱신(우리 과정)	매일	새 문서 수집→Lake→변환→임베딩→VectorDB

공통점 — 전부 「여러 단계가 순서대로」이고 「중간이 실패할 수 있다」 — 오케스트레이션이 필요한 조건

우리 파이프라인을 다섯 태스크로 — 역등한가

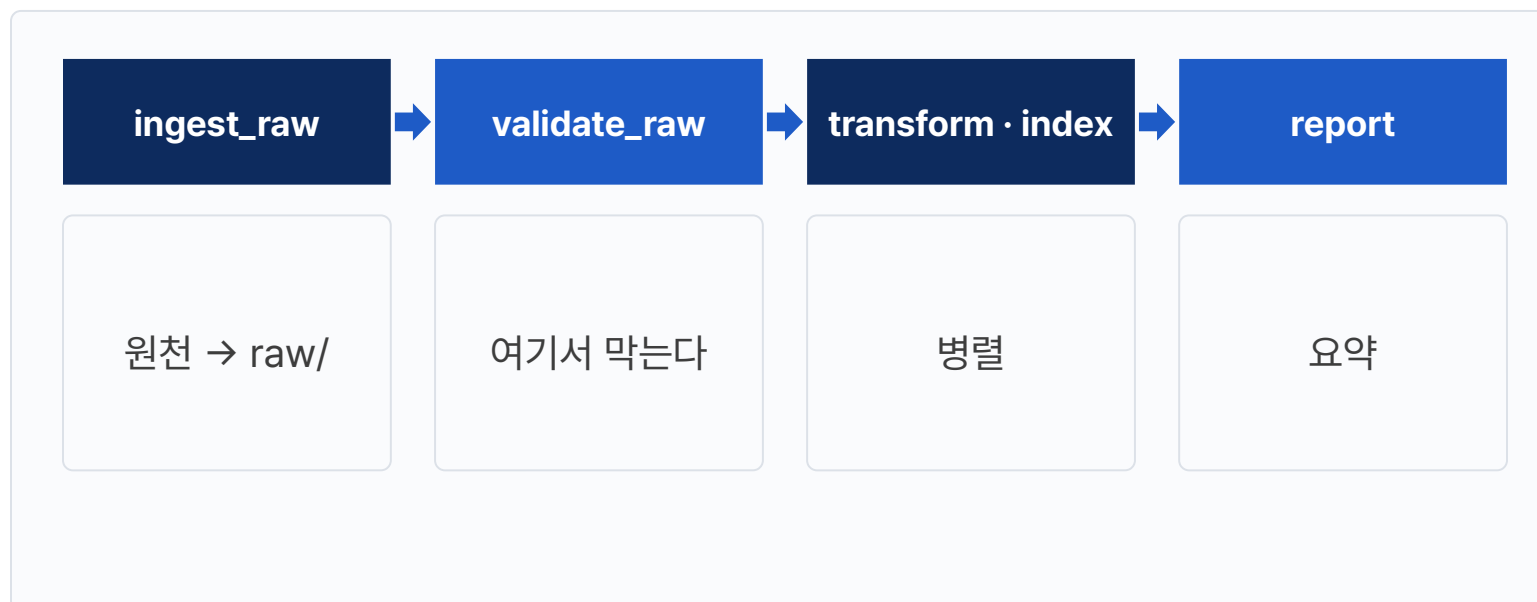
태스크	하는 일	역등한가
ingest_raw	원천에서 받아 MinIO raw/ 에 적재	같은 키에 덮어쓰는 → 예
validate_raw	건수·필수 컬럼 검사, 미달이면 실패	읽기만 → 예
transform_spark	Spark 로 정제해 Iceberg 테이블 갱신	MERGE → 예
build_index	문서를 잘라 임베딩해 Qdrant 갱신	uuid5 + 재적재 → 예
report	건수 요약을 남긴다	읽기만 → 예

각 태스크 옆에 「역등한가」를 반드시 적는다 — 아니오가 하나라도 있으면 재시도가 사고를 낸다

의존성 초안과 나눈 근거

의존성 초안

ingest >> validate >> [transform, index] >> report



VS

왜 이렇게 나뉘냐

예시일 뿐 — 팀마다 달라도 근거만 있으면 된다

- ingest·transform 분리 — 원천 API 가 불안정해 재시도가 잦다
- validate 를 따로 — 나쁜 데이터가 아래로 안 가게
- transform·build_index 는 서로 의존이 없어 병렬

골격 DAG — print 만 넣고 모양부터 맞춘다

1 STEPS 리스트 다섯 이름
ingest_raw · validate_raw · transform_spark · build_index · report

2 make(name) 이 print 함수를 돌려준다
"[{name}] 아직 내용 없음" — 내용은 8-3 에서 채운다

3 딕셔너리 컴프리헨션으로 태스크 5개
t = {s: PythonOperator(task_id=s, python_callable=make(s)) for s in STEPS}

4 의존성 한 줄
ingest >> validate >> [transform, index] >> report 를 t[...] 로

확인 포인트

**Graph 모양이
설계와 같은가**

unpause 를 빼고 trigger 만 — 회색에 머무는 이유

새 DAG 는 기본이 일시정지(paused) — 스케줄러가 태스크를 집어가지 않습니다

trigger 만	Grid 관찰	원인	unpause	dags show
unpause 없이 dags trigger	태스크가 계속 회색	DAG 가 paused 상태	dags unpause pipeline_skeleton	의존성을 글자로 확인

설계안을 문서로 — 8-3 의 출발점이자 8주차 산출물

- 1 표 — 태스크 · 하는 일 · 맥등 근거 · 재시도**
다섯 태스크 행을 채운다
- 2 의존성 — __ >> __ >> [__, __] >> __**
골격 DAG 의 마지막 줄과 같아야 한다
- 3 스케줄 · catchup: False**
학습 DAG 는 catchup=False 를 명시
- 4 나눈 근거 한 줄씩 · 실패했을 때 어디부터 다시 돌리나**
표를 채우고 근거를 적으면 오늘 몫이 끝

골격 파일은

지우지 말 것

8-3 설계도

이 1시간이 끝날 때 확인할 것

- 1 유즈케이스 다섯의 공통점을 한 문장으로 말했다
- 2 태스크 5개마다 「먹등한가」에 예가 적혀 있다
- 3 pipeline_skeleton 의 Graph 가 설계한 모양과 같다
- 4 unpause → trigger 뒤 Grid 가 순서대로 초록이 됐다
- 5 dags show 출력이 설계와 같다
- 6 분해안 표와 근거를 채웠다

8-2 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

DAG 작성 규칙을 한 장으로 모으고, 자주 겪는 오류 네 가지를 짚은 뒤 8-3 준비를 확인합니다

✓ DAG 선언·순서·재시도·CLI 요약표

✓ 체크리스트 스스로 확인

✓ DAG 가 안 보일 때·회색일 때·catchup

✓ 8-3 준비 — 골격 파일 · minio·qdrant

가져갈 것

분해안 문서와 pipeline_skeleton.py